Harness Engineering for Coding Agents

Instructions, state, scope, and verification: what each piece assumes about the model, and how to measure it

Module 1 · course repository `shipharness`

The problem: four failures to check for

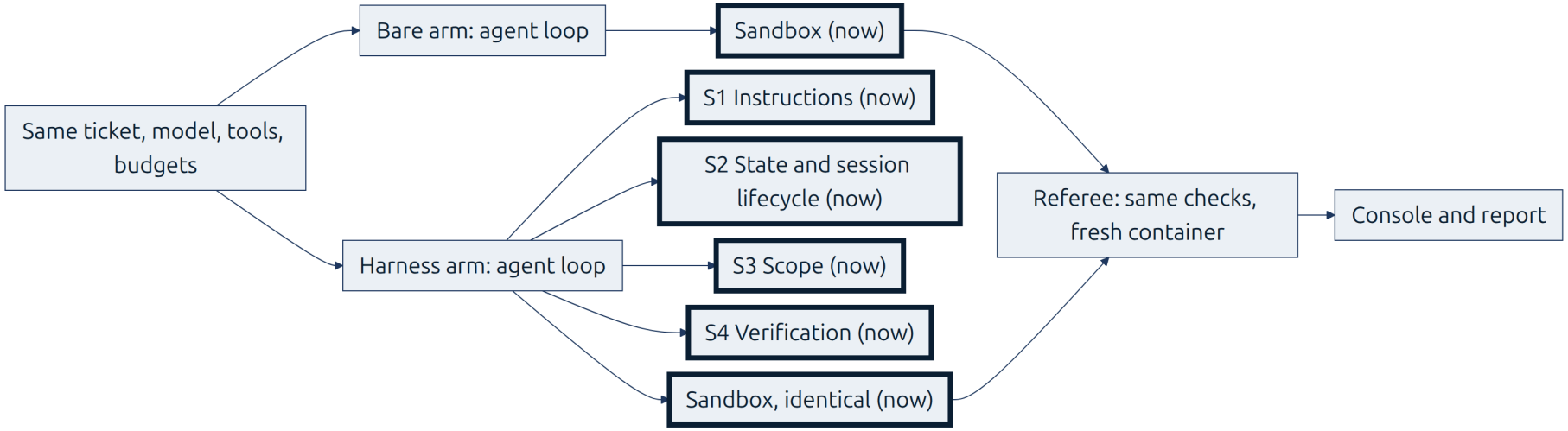
- Cross-tenant data: an Acme user sees Globex issues
- A second session that redoes or breaks the first
- Edits outside the ticket
- The agent's own tests pass while the behaviour is wrong
- The referee checks every run for all four, in both arms

Why it happens: four causes

- Cause 1: "anything it can’t access in-context while running effectively doesn’t exist" (A1, F-a-05)
- Cause 2: "each new session begins with no memory of what came before." (A2, F-a-08)
- Cause 3, side by side: A1 F-a-20; A4 F-a-18, F-a-19; A5 F-a-86; A6 F-a-21 (supplementary)
- Cause 4: "agents reliably skew positive when grading their own work" (A3, F-a-15); B13 F-b-45, F-b-50

A harness encodes assumptions about the model

- A1 on the work: "design environments, specify intent, and build feedback loops" (F-a-02)
- A3: "every component in a harness encodes an assumption about what the model can't do on its own" (F-a-124)
- Assumptions change with the model: A3 dropped context resets for Opus 4.5 (F-a-69)
- Four parts, S1 to S4, plus a sandbox that is identical for both arms
- The course's claim: measure each piece, keep what earns its place



shipharness, TeamTrack, console

- `shipharness`: run an agent with or without the harness, A/B it, judge it
- TeamTrack: multi-tenant issue tracker, orgs Acme and Globex, 25 API operations, 142 visible tests
- Console: five screens on keys `1` to `5`, `F` for full screen per arm

```
def run_and_judge(arm: Arm) -> tuple[AgentRunResult, ScoreRecord]:
    result = run_arm(task, specs[arm], deps, bus, run_dir)
    baseline.join()
    referee.baseline()  # computed by the thread (or now, if the thread failed)
    return result, _judge(referee, specs[arm], result, bus, run_dir)
try:
    with ThreadPoolExecutor(max_workers=2, thread_name_prefix="arm") as pool:
        futures = {arm: pool.submit(run_and_judge, arm) for arm in arms}
        done = {arm: future.result() for arm, future in futures.items()}
finally:
    ...
results = {arm: done[arm][0] for arm in arms}
```

src/shipharness/ab/demo.py

How we measure it

- Same agent, model, effort, task, repository, tools, sandbox, turn limit, cost cap; one deterministic referee
- House rules R1-R7: the course label for A1's "golden principles" (F-a-99)
- One primary measure per section, registered before any run; results are counts, never percentages
- Registered before the runs: no gap is reported as "no measurable difference on this task with this model"
- One piece at a time is A3's method: "removing one component at a time" (F-a-125)
- Datasets: 60 tasks in six groups, 40 test-generation targets

Check your setup

- Run `make doctor` now
- Run the `harness-setup` assessment on the bare TeamTrack
- Read the gaps it reports: instructions, state, scope, verification

Filled in live: `make doctor` output and the `harness-setup` assessment of the bare TeamTrack

Poll

- F1 org-wide issue search; rule at risk: R1 tenant isolation
- The pilot's trap ticket: a legacy handler without the tenant filter, next to the right pattern
- Of the pilot's 7 bare-arm F1 runs on that ticket, how many shipped at least one new R1 violation?
- A) 0-1 B) 2-3 C) 4-5 D) 6-7
- Close the poll before the launch

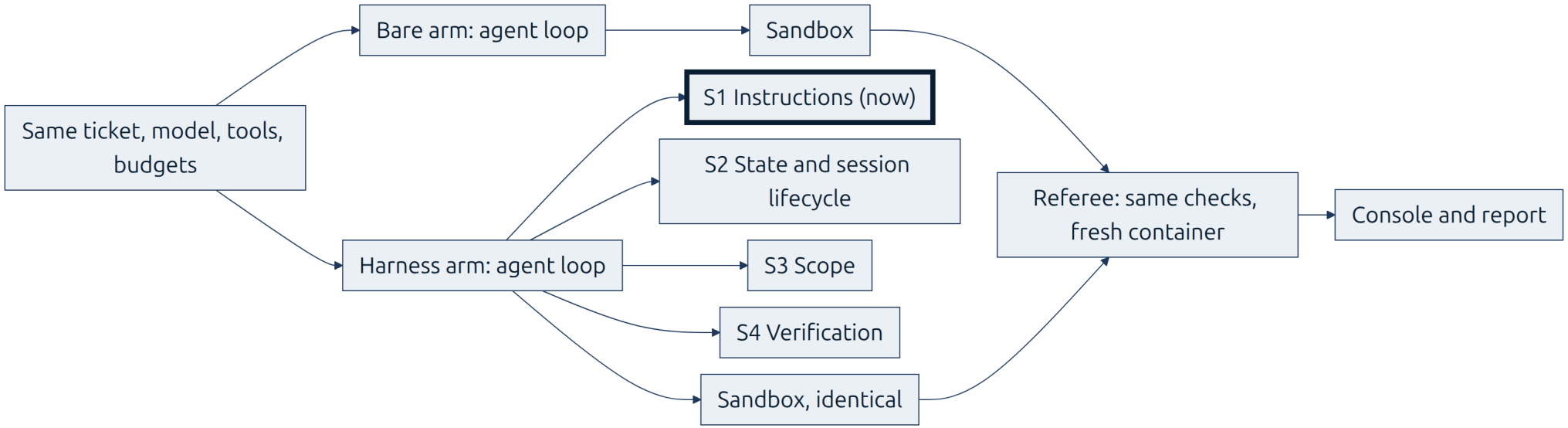
Filled in live: the room's poll result

Launching harness in action: F1 org-wide issue search

- Now: run `make show-1` in terminal B; it runs during slides 10-18, back on slide 19
- Harness in action: the harness arm alone on F1, live; not an A/B, so it stays out of the comparison
- Watch for: the agent reads the map, then only the documents the ticket needs
- Harness arm layers: instructions; the referee still judges the run (R1 probes, hidden tests, mutants)

S1: what the agent knows

- Explain why an agent does not apply a rule that exists only outside its context (A1)
- Structure knowledge as a short AGENTS.md map into `docs/`, plus one skill with progressive disclosure (B1)
- Read the S1 result: R1 and R3 failures with and without the instructions layer
- The assumption the map encodes: the model will not apply a rule it has not been pointed to



Problem: the rule is written down, the agent does not apply it

- F1 builds `GET /issues`, an org-wide issue search
- R1: another org's resource is 404, never 403
- The rule is in `docs/HANDBOOK.md`, and both arms can read it
- Nothing is withheld from the bare arm

Cause: what is not in context does not exist

- "anything it can’t access in-context while running effectively doesn’t exist" (A1, F-a-05)
- "A giant instruction file crowds out the task, the code, and the relevant docs" (A1, F-a-23)
- "Too much guidance becomes non-guidance." (A1, F-a-24)
- A long manual goes stale and cannot be checked mechanically (A1, F-a-25, F-a-26)
- Agents copy existing patterns, including uneven ones (A1, F-a-20)

Harness piece: AGENTS.md as a map

- "roughly 100 lines", "a map", "the table of contents" (A1, F-a-22, F-a-27, F-a-28)
- `docs/` is the system of record (A1, F-a-29, F-a-30)
- A1's layout: ARCHITECTURE.md at the root; SECURITY.md, RELIABILITY.md, QUALITY_SCORE.md, PLANS.md in docs/
- Also in docs/: design-docs/, product-specs/, references/ (A1, F-a-30, F-a-31)

Progressive disclosure

- Docs: "agents start with a small, stable entry point and are taught where to look next" (A1, F-a-36)
- Skills, three tiers: "Metadata (~100 tokens)", "Instructions (< 5000 tokens recommended)", "Resources (as needed)"
- Tiers from B1 and B2 (F-b-05, F-b-06)
- SKILL.md needs `name` and `description` (F-b-01 to F-b-04)

Keep the map true

- Docs are checked mechanically by linters and CI jobs (A1, F-a-37)
- Lint messages that teach: the error carries the fix into the agent's context (A1, F-a-97)
- Fairness: the bare arm gets every rule in `docs/HANDBOOK.md`
- A unit test checks that both forms carry each rule id

Code: the map

- The "Where to look" table of TeamTrack's AGENTS.md
- Each row: what you are about to do, and the document to read first

```
## Where to look
| When you are about to | Read first |
|---|---|
| add or change an endpoint | `.agents/skills/add-endpoint/SKILL.md` (step by step) |
| touch a router, service, repository, or model | `ARCHITECTURE.md` (layers, R6) |
| decide who may see or change data | `docs/SECURITY.md` (R1 tenant isolation, R2 roles) |
| shape a list, an error, or a status code | `docs/design-docs/api-conventions.md` (R3) |
| write a create endpoint or any write | `docs/RELIABILITY.md` (R4 idempotency, R5 audit) |
| change the database schema | `docs/RELIABILITY.md` (R7 migrations) |
| change issue or project behaviour | `docs/product-specs/teamtrack-base.md` |
| write or run tests | `docs/QUALITY_SCORE.md` (testing, known gaps) |
| plan a change or check the roadmap | `docs/PLANS.md` |
```

teamtrack/harness/instructions/AGENTS.md

Code: skills, tier 1 only

- `catalogue_block`: one line per skill, name and description
- The full SKILL.md is never pre-loaded; the agent reads it when the task matches

```
def catalogue_block(self) -> str:
    skills = self.catalogue()
    if not skills:
        return ""
    rows = [f"- {s.name}: {s.description} (read {s.path}/SKILL.md to use it)" for s in skills]
    return (
        "Available skills (load a skill's SKILL.md only when the task needs it):\n"
        + "\n".join(rows)
    )
```

src/shipharness/runner/skills_loader.py

Code: the stable prefix

- One stable text block: the map, then tier 1 of every skill
- The documents the map points to are not injected; the agent opens them

```
agents_md = strip_markers((source / MAP_FILE).read_text(encoding="utf-8")).strip()
blocks = [
    "Repository map (AGENTS.md at the workspace root). It points to the documents to read "
    "before you change code; open them with read_file when your task needs them.",
    agents_md,
]
catalogue = SkillsLoader(source / SKILLS_DIR, workspace_prefix=SKILLS_DIR).catalogue_block()
if catalogue:
    blocks.append(catalogue)
return "\n\n".join(blocks)
```

src/shipharness/harness/instructions/router.py

Demo block: harness in action, and a real run replayed

- `make show-1`, finished: screen 1 shows "primary verdict ready", then the full verdict
- Screen 1: the map read first, then the documents the ticket needs; screen 3: files read by category
- Real run, replayed: `make replay RUN=ab-20261016-212252-aa1644` (pilot run 3, F4)
- In that run the harness arm's change added 3 new R1 violations, and the referee's R1 check listed them

Filled in live: `make show-1`: the primary verdict and the documents the agent opened

Demo block: a constructed example, and what the pilot measured

- Constructed example: `make example-1`, scripted, no model; CONSTRUCTED EXAMPLE on every screen
- It shows the ⚠ other-org marks on a board, which no pilot arm produced
- Pilot, any new R1 violation fails: bare 0 of 7 failing, harness 0 of 7 (poll answer: A)
- No measurable difference on this task with this model: the bare arm applied R1 without the map
- Why: in run 2 every bare arm opened the handbook with its third tool call; no arm copied the legacy handler into a leak
- Post hoc, descriptive (10 runs per arm): done claimed by harness 5, bare 9; median cost $0.0122 against $0.0103

Runbook and next

- Runbook: `submodules/01-instructions/demo/README.md`
- Discussion: the map changed nothing on this ticket. What would have to be true of a model or a ticket for it to earn its place?
- Next: what happens when a session ends halfway

Poll

- F2 comments over two sessions; at risk: continuity and R7
- In the pilot both arms stopped after 12 turns and restarted with an empty context
- In those 7 A/Bs, which arms failed the continuity measure?
- A) No arm B) Only bare arms C) Only harness arms D) Some of both
- Close the poll before the launch

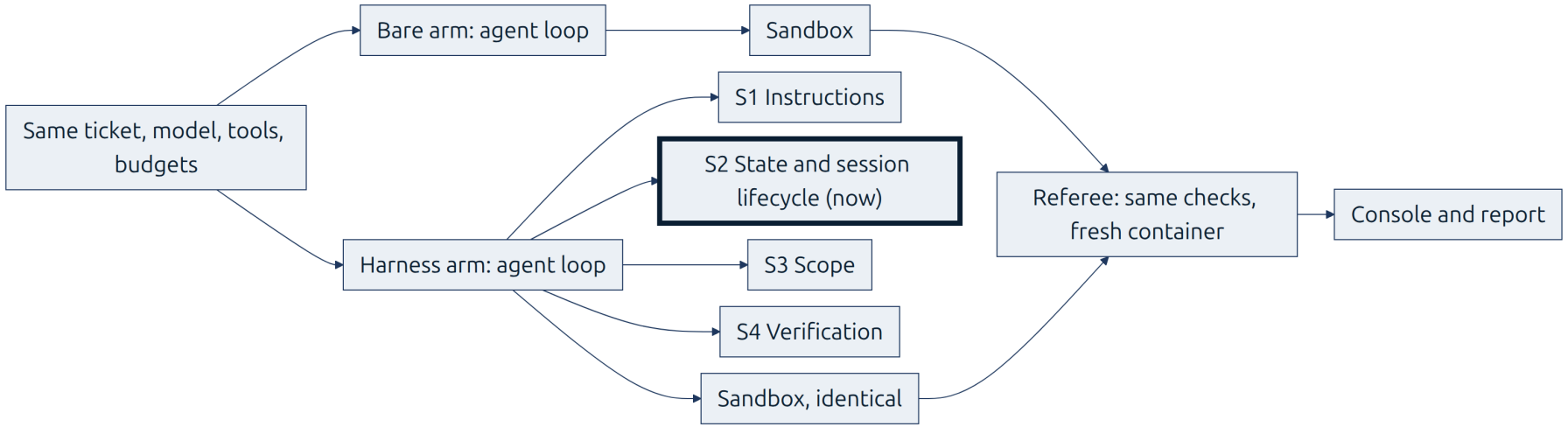
Filled in live: the room's poll result

Launching harness in action: F2 comments

- Now: run `make show-2 SESSION_BUDGET=12,38` in terminal B; it runs during slides 24-31, back on slide 32
- Harness in action: the harness arm alone on F2, first session 12 turns; not an A/B
- Watch for: the handoff written at the end of session 1, and session 2 resuming from it
- Harness arm layers: instructions, state

S2: what the agent remembers

- Describe the initializer and coding agents, `init.sh`, the progress file, and git history (A2)
- Design an exec plan (A1, A7) and a structured handoff (A3) that let a session resume after a reset
- Read the S2 result: redone or lost work and migration-head conflicts after a forced stop
- The assumption the state files encode: a fresh session cannot pick up the work from the repository alone



Cause: every session starts empty

- "each new session begins with no memory of what came before." (A2, F-a-08)
- "However, compaction isn’t sufficient." (A2, F-a-54, F-a-55)
- What the last session knew must be written down somewhere the next one reads

Harness piece: state on disk

- An initializer agent and a coding agent (A2, F-a-49, F-a-52)
- `init.sh` (F-a-50) and a progress file (F-a-50, F-a-53); the course names it `progress.md`
- Git history as state and as a way back (F-a-53, F-a-57)
- Course design: the initializer is a deterministic harness step

Getting up to speed, and ending a session

- A2's routine: `pwd`, progress and git log, the feature list, `init.sh`, a basic test (F-a-58 to F-a-62)
- "End the session by writing a git commit and progress update." (F-a-63)
- Both steps are the same every session

Exec plans and the structured handoff

- Exec plans move from active to completed (A1, A7; F-a-34, F-a-35, F-a-45 to F-a-48)
- A3's context reset: a fresh agent "combined with a structured handoff" (F-a-65)
- The course mechanism is the session turn budget, the same for both arms
- A reset has a price: "orchestration complexity, token overhead, and latency" (A3, F-a-68)
- "Opus 4.5 largely removed that behavior on its own, so I was able to drop context resets from this harness entirely." (A3, F-a-69)

Code: init.sh

- Migrations first: house rule R7, exactly one head
- Then a basic end-to-end test that stops at the first failure

```
echo "== migrations (house rule R7: exactly one head)"
python -m alembic heads
python -m alembic upgrade head

echo "== basic end-to-end test: issues, stop at the first failure (full suite: run_visible_tests)"
python -m pytest tests/visible/test_issues.py -x -q --no-header -p no:cacheprovider
```

src/shipharness/harness/state/assets/init.sh

Code: stop and resume

- `resume_text`: extra resume text for the harness arm
- The neutral resume sentence comes first, the same for both arms

```
def resume_text(summary: str, phase: str = "build") -> str:
    """Extra resume text for the harness arm (the neutral resume sentence comes first)."""
    text = (
        "Start with the Getting up to speed steps before you change code. The structured "
        "handoff from the last session is in session-handoff.md."
    )
    if phase == "test":
        text = f"{text} {TEST_PHASE_FIRST}"
    return f"{text} Its machine summary:\n\n{summary[:3000]}" if summary else text
```

src/shipharness/harness/state/lifecycle.py

Code: the session log

- `session_entry`: one log line per session
- Index, start and end commit, how the session ended, and a summary

```
def session_entry(
    result: SessionResult, files_changed: int, tests: str, claim_summary: str = ""
) -> str:
    """One log line: index, start/end commit, how the session ended, and a summary."""
    entry = (
        f"- Session {result.index} ({result.phase}): start {result.start_sha[:7] or 'none'}, "
        f"end {result.end_sha[:7] or 'none'}, ended by {result.ended_by} after "
        f"{result.turns} turns. {files_changed} files changed; visible tests {tests}."
    )
    summary = " ".join(mask_keys(claim_summary).split())[:200]
    return entry + (f' Agent summary: "{summary}"' if summary else "")
```

src/shipharness/harness/state/progress.py

Demo block: harness in action, and a real run replayed

- `make show-2`, finished: screen 1 shows the session stop, the handoff, and the resume
- Screen 3: state files read at the start of session 2; screen 5: the trace links
- Real run, replayed: `make replay RUN=ab-20261016-213317-9a0ec1` (pilot run 3, F2)
- Its harness arm called `finish(claim="not_done")` after session 1; the build phase ended, session 2 never ran
- The fix, for both arms: before the last build session, that call ends only the session

Filled in live: `make show-2`: the primary verdict and the file session 2 read first

Demo block: a constructed example, and what the pilot measured

- Constructed example: `make example-2`, scripted, no model; CONSTRUCTED EXAMPLE on every screen
- Pilot, continuity failure fails: bare 0 of 7, harness 0 of 7 (poll answer: A); no measurable difference on this task with this model
- The measure cannot fail when session 1 leaves nothing to redo: in runs 3 and 4, 6 of 12 arms built nothing
- The bare arm found its own state: in run 2 its session 2 ran `git status` in its fourth tool call
- Post hoc, descriptive (9 runs per arm): agent time limit hit, harness 4, bare 0; median wall 361 s against 232 s
- Class-profile round, with the fix: both arms built the feature, 56 of 56 hidden tests

Runbook and next

- Runbook: `submodules/02-state-lifecycle/demo/README.md`
- Discussion: the bare arm found its own state with `git status`. Which state file would you keep for this model, and how would you measure it?
- Next: a 10-minute break

Break

Back in 10 minutes

Filled in live: return time

Poll

- F3 idempotency keys; at risk: scope
- The ticket: "shared middleware is out of scope"; tests in a separate test phase, no app code changes
- Of the pilot's 7 bare-arm F3 runs, how many touched a file outside the allowed paths, or changed source in the test phase?
- (a) 0-1 (b) 2-3 (c) 4-5 (d) 6-7
- Close the poll before the launch

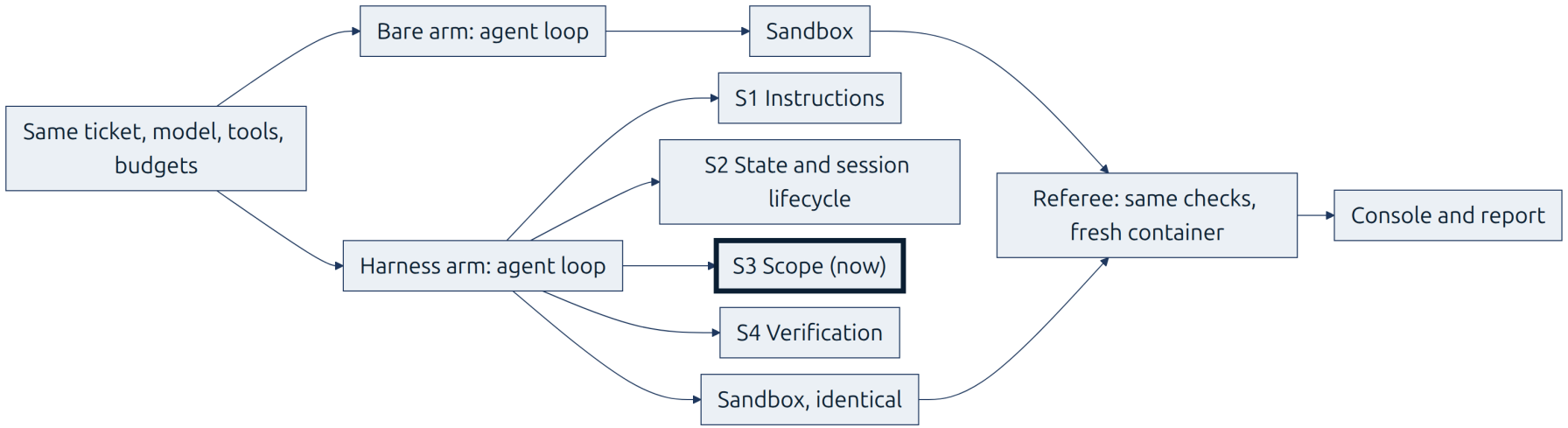
Filled in live: the room's poll result

Launching harness in action: F3 idempotency keys

- Now: run `make show-3` in terminal B; it runs during slides 38-45, back on slide 46
- Harness in action: the harness arm alone on F3; not an A/B
- Watch for: a blocked or reverted edit with its reason, and the sandbox policy at work
- Build, then a test phase; allow lists from the task; shared middleware is off-limits
- Harness arm layers: instructions, state, scope

S3: what the agent may change and reach

- Enforce one feature at a time, allowed paths per phase, and a `passes` field only the harness sets
- Explain the container limits for both arms and read a `sandbox_denied` event
- Read the S3 result: out-of-scope files and source edits during the test phase
- The assumption the scope layer encodes: the agent will change files outside the ticket unless code stops it



Cause: copied patterns and hard-to-undo actions

- "Codex replicates patterns that already exist in the repository—even uneven or suboptimal ones" (A1, F-a-20)
- A4: "(Run this in a container, not your actual machine)"; the agent ran `pkill -9 bash` (F-a-18, F-a-19)
- A5: file operations "to the project directory only" (F-a-86)
- A6, supplementary: actions "that are difficult to reverse or affect shared systems" (F-a-21)

Harness piece: one feature list, one feature

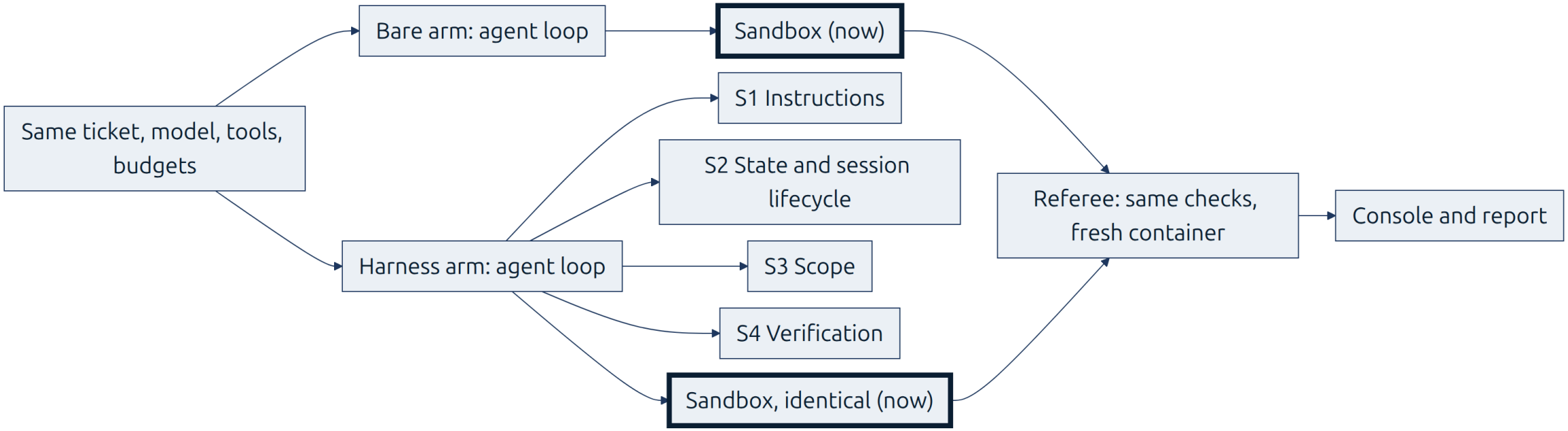
- `feature_list.json` as the source of truth; JSON over Markdown (A2, A5; F-a-78, F-a-81, F-a-85)
- The `passes` field (F-a-79, F-a-82); one feature at a time (F-a-83)
- "It is unacceptable to remove or edit tests because this could lead to missing or buggy functionality." (F-a-80)
- Course design: only the harness sets `passes`, with `evidence` and `testedAt` (A2 F-a-108, A3 F-a-110)

Allowed paths per phase

- Build and test phases each have an allow list
- A patch outside the list is rejected
- A command's change outside the list is reverted
- The agent reads the reason
- Composite citation: A5 F-a-86, F-a-87; A1 F-a-92, F-a-98; A2 F-a-57

Sandbox 1: why a container

- A4: "for each agent, a Docker container is spun up", and no internet access (F-a-88, F-a-90)
- A5's three-layer security model (F-a-86, F-a-87)
- Codex: "By default, the agent runs with network access turned off." (D21-D23)



Sandbox 2: the same limits for both arms

- `--network none`, memory without swap, 256 processes, non-root (D24-D30)
- Read-only root; only `/work` and `/tmp` writable; all capabilities dropped
- `sandbox_denied` events carry a reason code
- Never an A/B variable, so the pilot cannot tell whether it earns its place
- Kept on the sources' advice: a container per agent, no internet (A4, F-a-88, F-a-90); project directory only (A5, F-a-86)

Code: allowed paths and evidence

- The three `if` checks of `reason()`: harness-only files, state files, the allow list
- `passes` is set only when the evidence is there

```
if rel in HARNESS_ONLY:
    return HARNESS_ONLY[rel]
if self.state_files and any(glob_matches(rel, p) for p in STATE_FILES):
    return None
if self.allowed is None or any(glob_matches(rel, p) for p in self.allowed):
    return None
```

src/shipharness/harness/scope/guard.py

```
if evidence is None:
    message = f"passes stays false for {feature['id']}: {why_not}"
    return [HarnessDecision(kind="note", layer="scope", message=message)]
tested_at = now().astimezone(UTC).strftime("%Y-%m-%dT%H:%M:%SZ")
feature.update(passes=True, evidence=evidence, testedAt=tested_at)
```

src/shipharness/harness/scope/status.py

Code: the sandbox policy

- `SandboxPolicy`: identical for both arms in every run
- Network, CPU, memory, swap, processes, user, root, tmpfs, capabilities

```
network: str = "none"  # only loopback inside the container
cpus: float = 1.0
memory_mb: int = 1024
swap_mb: int = 0  # --memory-swap equal to --memory: no swap
pids_limit: int = 256
user: str = "1000:1000"  # non-root
read_only_root: bool = True  # writes only to /work (workspace) and /tmp
tmpfs: dict[str, str] = {"/tmp": "rw,nosuid,size=256m"}
cap_drop: tuple[str, ...] = ("ALL",)
no_new_privileges: bool = True
```

src/shipharness/sandbox/policy.py

Demo block: harness in action, and a real run replayed

- `make show-3`, finished: screen 1 lists blocked or reverted decisions with their reasons
- Screen 3: the diff map and any `sandbox_denied` event
- Real run, replayed: `make replay RUN=ab-20261016-214151-ca9841` (pilot run 3, F4)
- Its bare arm changed 2 files outside the allowed paths; its harness arm, with the scope layer, 0

Filled in live: `make show-3`: blocked or reverted decisions and any `sandbox_denied` event

Demo block: a constructed example, the sandbox, and what the pilot measured

- Constructed example: `make example-3`, scripted, no model; CONSTRUCTED EXAMPLE on every screen
- Sandbox moment: no pilot A/B recorded a `sandbox_denied` event; show the policy printout
- Pilot, out-of-scope file or test-phase source edit fails: bare 0 of 7, harness 0 of 7 (poll answer: a); no measurable difference on this task with this model
- Post hoc, F3 (9 runs per arm): harness arms shipped a median of 0 tests against 6; claimed done in 4 against 9
- Post hoc, F4 (all four layers): bare arms left the allowed paths in 6 of 6 runs, harness arms in 0 of 6
- A measure nobody registered for scope: a reason to register it next time, not a result

Runbook and next

- Runbook: `submodules/03-scope-sandbox/demo/README.md`
- Discussion: the scope layer's clearest effect showed where it was not the primary measure. How would you register that before the next run?
- Next: who decides that the work is done

Poll

- F4 optimistic concurrency; at stake: hidden tests (69) and referee mutants (4)
- A run passes only with all 69 hidden tests passing and all 4 mutants killed
- In the pilot's 4 F4 A/Bs, which arm passed more often?
- A) Harness arm B) Bare arm C) Both, equally often D) Neither arm passed once
- The final feature poll follows on slide 50

Filled in live: the room's poll result

Poll: the final feature

- F5 Bulk close: one session of 30 turns; 46 hidden tests, 3 referee mutants
- F6 Watchers: two sessions, so a context reset; 54 hidden tests, 3 mutants
- F7 Labels: two sessions, so a context reset; 64 hidden tests, 3 mutants
- The winner runs as tonight's one live A/B, and the runner-up after it if the session figure allows
- Close the poll before the launch

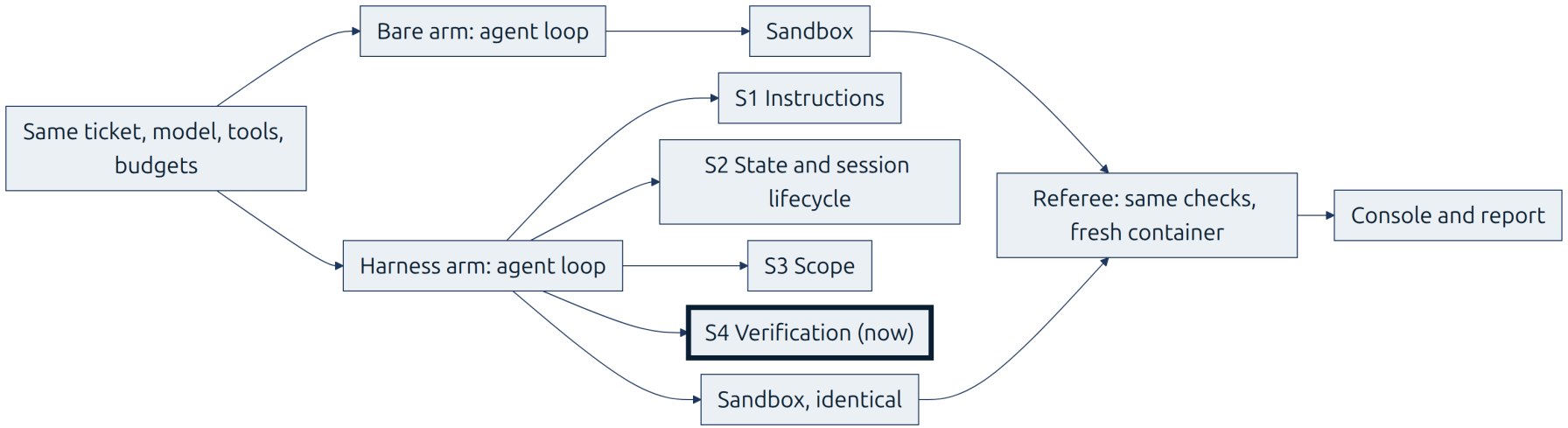
Filled in live: the room's poll result

Launching harness in action on F4, and the final A/B

- Now: `make show-4` in terminal B, then `make final FEATURES=<poll winner>` in terminal C
- show-4 runs during slides 52-60, back on slide 61; the final returns on slides 64-67
- show-4: the harness arm alone on F4; watch the handoffs, the test phase, and the filters
- The final is tonight's one live A/B: both arms, all four layers, blinded reviewer on both diffs
- Class profile for both arms; final run cap $0.25 per arm

S4: what proves the work is done

- Build verification the agent does not control: filters, mutants, house-rule checks, held-out tests, reviewer, gate
- Find the first wrong turn in a trace
- Read a fair A/B: hidden tests passed and mutants caught, as counts per run
- The assumption the gate and filters encode: the agent's "done" and its own tests cannot be trusted



Cause: self-grading and visible tests

- "agents reliably skew positive when grading their own work" (A3, F-a-15)
- "while every frontier agent saturates the visible suite, reward hacking persists" (B13, F-b-45)
- Mostly features that pass alone and fail when combined (B13, F-b-50)
- "it is easy to see tests pass and assume the job is done, when this is rarely the case." (A4, F-a-17)

Harness piece: test filters

- Filters 1-3 from TestGen-LLM: builds, passes reliably, adds coverage (B11, F-b-16 to F-b-22)
- Passes reliably: "every one of five executions" (B11, F-b-19); the referee also re-runs shipped tests five times
- Filters 4 and 5 are course filters: real assertions, no source change in the test phase
- A test is kept only if it passes every filter

Mutants and hidden tests

- ACH: mutants for the fault of concern, then hardening (B12, F-b-29, F-b-32)
- 49% of tests that uniquely kill a mutant add no line coverage (F-b-40)
- Filter-set and referee-set mutants never overlap (course design)
- Hidden tests that compose features (B13, B14; F-b-42 to F-b-44)
- 441 hidden tests across F1-F7 and controls; 69 for F4

A separate evaluator and a gate

- Blinded reviewer with grading criteria (A3, F-a-110 to F-a-112)
- Clean-state gate (A2, F-a-84; A4, F-a-127)
- House rules R1-R7 (course label)
- A3: "It is worth the cost when the task sits beyond what the current model does reliably solo." (F-a-123)

Measure each piece

- Read the trace: find the first wrong turn (A3, F-a-126); screen 5 shows it per arm, as a heuristic
- Measure one piece: "removing one component at a time and reviewing what impact it had on the final result." (A3, F-a-125)
- In shipharness: `make demo-N LAYERS=-<layer>` runs one A/B without that layer
- A section's A/B tests the stack so far; only taking one piece out tests that piece

Code: the filter pipeline

- Each proposed test goes through the filters in order
- The first failure discards it, with a reason; later filters never run

```
for flt in filters:
    try:
        result = flt.evaluate(cand, ctx)
    except Exception as exc:  # noqa: BLE001 - a crashing filter discards, with a reason
    ...
    if not result.reason:
        result = result.model_copy(update={"reason": f"filter {flt.number}: no reason"})
    results.append(result)
    if bus is not None:
        bus.publish("test_filter_result", result, run_id=run_id, arm=arm)
    if not result.passed:
        break  # discarded: later (more expensive) filters never run
```

src/shipharness/filters/pipeline.py

Code: a tenant probe and blinding

- `judge`: compare one response with what tenant isolation requires
- A foreign path must return 404; anything else fails

```
def judge(probe: Probe, response: Any, t: Target, home: str, other: str) -> Finding | None:
    """Compare one response with what tenant isolation requires."""
    status, body = response.status_code, json_body(response)
    ok = 200 <= status < 300
    leaked = ok and (
        t.adapter.is_foreign(body, other)
        or (probe.kind == "anonymous" and t.adapter.is_foreign(body, home))
    )
    if probe.kind in ("foreign_path",) or probe.kind.startswith("foreign_param:"):
        failed = status != 404
    else:  # no_path, foreign_body, anonymous: success itself is the failure (no_path: only leaks)
        failed = leaked if probe.kind == "no_path" else ok
```

src/shipharness/checks/r1_tenant.py

Code: the gate

- Verification the agent does not control
- Visible tests, the build-phase checks, the clean-state check
- The gate passes only if no item fails

```
names = ["visible tests", *(_ITEM_ORDER if phase == "build" else []), "clean state"]
...
if phase == "build":
    check_items, check_lines = _check_items(sandbox, cfg, root, on_progress)
    items += sorted(check_items, key=_order)
    lines += check_lines
clean = clean_state_check(sandbox, base_sha, cfg.limit(120))
items.append(clean)
if clean.status == "fail":
    lines.append(f"FAIL clean state: {clean.detail}")
    ...
passed = not any(i.status == "fail" for i in items)
```

src/shipharness/harness/verify/gate.py

Demo block: harness in action, and a real run replayed

- `make show-4`, finished: screen 1 shows the gate pills ✓ or ✗ and the gate feedback
- Screen 3: the filter table with discard reasons, the mutant table, the referee counters
- Real run, replayed: `make replay RUN=ab-20261017-090903-926c02` (class-profile round, F4)
- Its bare arm claimed done with 68 of 69 hidden tests passing; the referee failed the run

Filled in live: `make show-4`: the gate pills and the filter table

Demo block: a constructed example, and what the pilot measured

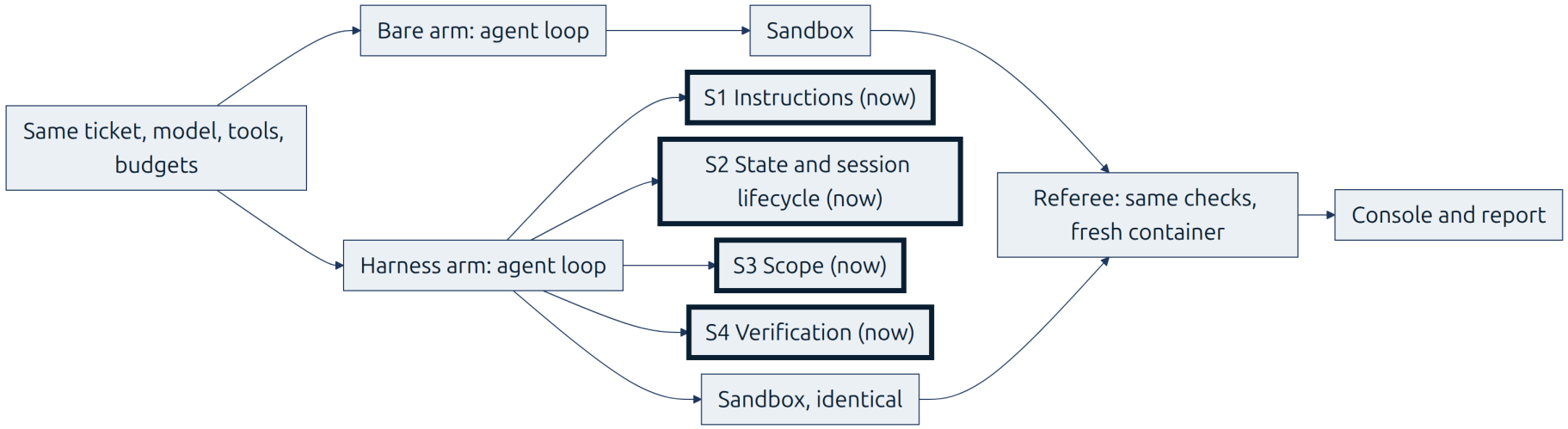
- Constructed example: `make example-4`, scripted, no model; CONSTRUCTED EXAMPLE on every screen
- Pilot, all 69 hidden tests and all 4 mutants: bare 0 of 4 passing, harness 0 of 4 (poll answer: D); no measurable difference on this task with this model
- Run 3: every bare arm hit the agent time limit; the harness arms shipped no tests, so killed 0 of 4 mutants
- Class-profile round: both arms 68 of 69 hidden tests; mutants killed: bare 4, harness 1
- Post hoc, runs 2-4 (34 runs per arm): no done claim came with failing hidden tests (0 of 19 bare, 0 of 10 harness)
- The class-profile bare arm on slide 61 is the first done claim with a failing hidden test since

Runbook and next

- Runbook: `submodules/04-verification/demo/README.md`
- Discussion: one false "done" in 4 pilot F4 runs per arm, from the bare arm. What would a gate need to catch to earn its turns on this model?
- Next: the final results

Final results: the A/B lines

- The poll winner, and the runner-up if it ran; all four layers; class profile
- Run cap $0.25 per arm
- No pre-registered primary measure: read hidden tests and mutants per arm
- One A/B is one dot: read it against the pilot's dots



Filled in live: terminal C's arm lines and session figure

Final results: console

- Screen 1: the turns, harness decisions, gate pills, "primary verdict ready"
- Screen 3: evidence, filters, mutants, reviewer verdict
- Screen 5: the first wrong turn and the trace links

Filled in live: screens 1, 3, and 5 of the finished final A/B, both arms

Final results: boards

- Screen 2: each arm's running board as an Acme user
- Look for ⚠ other-org and ⧉ duplicate marks

Filled in live: both boards side by side

Final results: screen 4

- Screen 4: one dot per recorded run, wins and ties alike
- Tonight's final adds one dot per arm
- Filled dot = passed, hollow = failed: never colour alone

```
fill = f"var(--{arm})" if dot["passed"] else "var(--surface)"
tip = f"{dot['run_id']}: {dot['value']}" + (
    f" of {dot['total']}" if dot["total"] else ""
)
parts.append(
    f'<circle cx="{x(dot["value"]):.1f}" cy="{y - stack * 10}" r="5" '
    f'fill="{fill}" stroke="var(--{arm})" stroke-width="2">'
    f"<title>{html.escape(tip)} ({dot['status']})</title></circle>"
```

src/shipharness/ab/report.py

Filled in live: tonight's final on screen 4

What the pilot measured: every section

Primary measure, runs passing per arm: pilot runs 3 and 4 and the class-profile round

- S1, F1, no new R1 violation: bare 7 of 7, harness 7 of 7
- S2, F2, no continuity failure: bare 7 of 7, harness 7 of 7
- S3, F3, no out-of-scope file or test-phase source edit: bare 7 of 7, harness 7 of 7
- S4, F4, all hidden tests and mutants: bare 0 of 4, harness 0 of 4
- All four: no measurable difference on this task with this model; the gate did not pass, so the planned larger experiments were not run
- Post hoc (34 runs per arm): outside the allowed paths, harness 0, bare 9; done claimed 10 against 19, none false

Assignments

- A1 Instructions: `submodules/01-instructions/assignment/`, ≤ $2.50 and 150 traces
- A2 State and session lifecycle: `submodules/02-state-lifecycle/assignment/`, ≤ $1.50 and 60 traces
- A3 Scope and sandbox: `submodules/03-scope-sandbox/assignment/`, ≤ $0.25 and 30 traces
- A4 Verification and portfolio: `submodules/04-verification/assignment/`, ≤ $1.75 and 120 traces
- Each: a brief with grading criteria, a starter folder, `make check-assignment-N` offline, one live step
- Total ≤ $6 and ≤ 400 traces; Thursday review

Skills and the starter kit to take home

- `harness-setup`, `assured-tests`, `house-rules`, `skill-template`, in the Agent Skills format (B1)
- `uv run shipharness init --target <repo>`: the kit, with TeamTrack's values and REPLACE markers
- `uv run shipharness init --check <repo>` lists the markers still to fill; no model calls
- Guide: `docs/apply-to-your-repo.md`, ending with which layers earn their place in your repository

```
name: skill-template
description: >
  Starting point for writing a new Agent Skill: frontmatter with the spec's fields and a guide to
  the sections of the body. Use when creating a new skill: copy this folder, rename it, set name to
  the new folder name, and replace every placeholder.
license: MIT
```

`skills/skill-template/SKILL.md`

References (1 of 2)

- A1 Harness engineering: leveraging Codex in an agent-first world · Ryan Lopopolo, OpenAI · 2026-02-11
- A2 Effective harnesses for long-running agents · Justin Young, Anthropic · 2025-11-26
- A3 Harness design for long-running application development · Prithvi Rajasekaran, Anthropic · 2026-03-24
- A4 Building a C compiler with a team of parallel Claudes · Nicholas Carlini, Anthropic · 2026-02-05
- A5 Autonomous Coding Agent Demo (README) · Anthropic · 2025-11-26
- A6 Prompting best practices · Anthropic, Claude docs · living page; A7 Using PLANS.md · Aaron Friel, OpenAI · 2025-10-07

References (2 of 2)

- B1 Agent Skills Specification; B2 What are skills?; B3 How to add skills support · Agent Skills project
- B4-B6 agentskills repo and skills-ref; B7 Claude Code skills; B8 Codex skills; B9-B10 Cursor skills
- B11 TestGen-LLM · Alshahwan et al., Meta · 2024; B12 ACH · Foster et al., Meta · 2025
- B13 SpecBench · Zhao, Srikanth, Wu, Jiang (Weco AI) · 2026; B14 WecoAI/SpecBench repo
- B15 (Spec-Bench, speculative decoding) is unrelated and not cited